Preprocessing dan FE untuk Klasifikasi Keterlambatan Keberangkatan

Linda Sofiana - 25.51.1762 | M. Nizar Farizi - 25.51.1805

Penjelasan disetiap block function untuk menjawab `WHY`, pada kasus Flight Delay Classification yang akan digunakan oleh model yang sudah dipilih.

Peta alur

`Pemilihan kolom → cleaning dan target → FE → split waktu → pilih masukan model → fit pada train → transform validation/test → matriks X dan label y`

Bagian	Alasan utama	Mempelajari statistik data?
`numeric`	Menangani nilai numerik tidak valid secara konsisten	Tidak
`clean_chunk`	Memastikan populasi/label klasifikasi keterlambatan keberangkatan dan format fitur sesuai	Tidak
`engineer`	Mewakili siklus waktu dan ketidaktersediaan nilai	Tidak
`make_preprocessor`	Menyusun aturan transformasi menurut jenis fitur	Belum; baru saat fit
`model_input`	Memisahkan fitur dari target dan metadata	Tidak
`run`: split dan fit/transform	Mempertahankan pemisahan informasi train dan evaluasi	Hanya saat fit pada train

Konfigurasi yang digunakan

`CANCELLED` dan `DEP_DELAY` dibaca untuk menentukan populasi/target. Keduanya tidak masuk `MODEL_FEATURES`. `FL_DATE` dan `SOURCE_ROW_NUMBER` dipertahankan sebagai metadata. Bulan, hari, dan menit jadwal akhirnya direpresentasikan sebagai sin/cos.

```
ROOT = Path(__file__).resolve().parent
SOURCE = ROOT.parent / "flights_sample_3m.csv"
CHUNK_SIZE = 100_000
INPUT_COLUMNS = [
    "FL_DATE", "AIRLINE_CODE", "ORIGIN", "DEST", "CRS_DEP_TIME", "CRS_ARR_TIME",
    "CRS_ELAPSED_TIME", "DISTANCE", "CANCELLED", "DEP_DELAY",
]
FEATURES = [
    "AIRLINE_CODE", "ORIGIN", "DEST", "MONTH", "DAY_OF_WEEK",
    "SCHEDULED_DEP_MINUTES", "SCHEDULED_ARR_MINUTES", "CRS_ELAPSED_TIME",
"DISTANCE",
]
TARGET = "DELAYED_15"
```

```
METADATA = ["SOURCE_ROW_NUMBER", "FL_DATE"]
OUTPUT_COLUMNS = METADATA + FEATURES + [TARGET]
SPLITS = {
    "train": ("2019-01-01", "2021-12-31"),
    "validation": ("2022-01-01", "2022-12-31"),
    "test": ("2023-01-01", "2023-08-31"),
}
CATEGORICAL = ["AIRLINE_CODE", "ORIGIN", "DEST"]
CONTINUOUS = ["CRS_ELAPSED_TIME", "DISTANCE"]
CYCLES = {
    "MONTH": (12, 1), "DAY_OF_WEEK": (7, 0),
    "SCHEDULED_DEP_MINUTES": (1440, 0), "SCHEDULED_ARR_MINUTES": (1440, 0),
}
CYCLIC_FEATURES = [f"{column}_{suffix}" for column in CYCLES for suffix in ("SIN",
"COS")]
MISSING_FLAGS = [f"{column}_MISSING" for column in
                 ["SCHEDULED_DEP_MINUTES", "SCHEDULED_ARR_MINUTES"] + CONTINUOUS]
MODEL_FEATURES = CATEGORICAL + CONTINUOUS + CYCLIC_FEATURES + MISSING_FLAGS
PREPARED_COLUMNS = OUTPUT_COLUMNS + CYCLIC_FEATURES + MISSING_FLAGS
```

1. Konversi nilai numerik

Sumber: `numeric`, baris 64–66 pada versi saat dokumentasi dibuat.

```
# Sumber: e1-data-preparation.py, baris 64–66.
# WHY: Input numerik dari CSV dapat berisi teks yang tidak bisa dikonversi atau
infinity.
# Menyamakan keduanya menjadi missing membuat aturan penanganannya konsisten.
# Missing pada target akan dikeluarkan oleh clean_chunk; missing pada fitur dapat
# ditangani sesuai jenis fitur. Tidak ada nilai pengisi yang dipelajari di sini.
def numeric(series):
    result = pd.to_numeric(series, errors="coerce")
    return result.where(np.isfinite(result))
```

2. Cleaning, populasi klasifikasi keterlambatan keberangkatan, dan target

Sumber: `clean_chunk`, baris 69–115 pada versi saat dokumentasi dibuat.

```
# Sumber: e1-data-preparation.py, baris 69–115.
# WHY: Klasifikasi keterlambatan keberangkatan memprediksi apakah penerbangan
# non-cancelled mengalami keterlambatan keberangkatan minimal 15 menit.
# Pembatalan atau label yang hilang tidak boleh berubah menjadi kelas tidak
terlambat.
# Tanggal valid diperlukan agar pemisahan masa lalu dan masa depan dapat
dilakukan.
# Normalisasi kategori mencegah spasi/huruf berbeda membentuk kategori semu.
# HHMM diubah ke menit karena selisih angka HHMM bukan selisih waktu yang benar.
# Nilai fitur tidak valid dibuat missing tanpa mengisi dari statistik seluruh
dataset.
```

```
# Delay negatif/ekstrem tetap dipertahankan karena keduanya dapat merupakan
kejadian sah.
# Nomor record dan hitungan pengecualian memungkinkan hasil ditelusuri ke sumber.
def clean_chunk(frame, start_row):
    """Aturan tetap; tidak mempelajari statistik dari train/validation/test."""
    stats = Counter(input_rows=len(frame))
    dates = pd.to_datetime(frame.FL_DATE, format="%Y-%m-%d", errors="coerce")
    cancelled, delay = numeric(frame.CANCELLED), numeric(frame.DEP_DELAY)
    # Pengecualian saling eksklusif agar jumlah baris dapat direkonsiliasi.
    valid = pd.Series(True, index=frame.index)
    for reason, condition in [
        ("excluded_invalid_cancelled", ~cancelled.isin([0, 1])),
        ("excluded_cancelled", cancelled.eq(1)),
        ("excluded_missing_or_invalid_target", delay.isna()),
        ("excluded_invalid_date", dates.isna()),
        ("excluded_outside_split_period", ~dates.between("2019-01-01", "2023-08-
31")),
    ]:
        rejected = valid & condition
        stats[reason] += int(rejected.sum())
        valid &= ~rejected
    kept = frame.loc[valid]
    dates = dates.loc[valid]
    result = pd.DataFrame(index=kept.index)
    result["SOURCE_ROW_NUMBER"] = pd.Series(np.arange(start_row, start_row +
len(frame)), index=frame.index).loc[valid]
    result["FL_DATE"] = dates.dt.strftime("%Y-%m-%d")
    for column in ("AIRLINE_CODE", "ORIGIN", "DEST"):
        original = kept[column]
        cleaned = original.str.strip().str.upper().replace("", pd.NA)
        stats[column + "_normalized"] += int((original.fillna("") !=
cleaned.fillna("")).sum())
        result[column] = cleaned
    result["MONTH"] = dates.dt.month.astype("int8")
    result["DAY_OF_WEEK"] = dates.dt.dayofweek.astype("int8")
    for source, destination in [("CRS_DEP_TIME", "SCHEDULED_DEP_MINUTES"),
("CRS_ARR_TIME", "SCHEDULED_ARR_MINUTES")]:
        time = numeric(kept[source])
        valid_time = (time.between(0, 2359) & time.mod(100).lt(60) &
time.mod(1).eq(0)).fillna(False)
        midnight = time.eq(2400).fillna(False)
        stats[source + "_2400_normalized"] += int(midnight.sum())
        stats[source + "_invalid_to_missing"] += int((kept[source].notna() & ~
(valid_time | midnight)).sum())
        result[destination] = ((time // 100) * 60 + time %
100).where(valid_time).mask(midnight, 0)
    for column in ("CRS_ELAPSED_TIME", "DISTANCE"):
        value = numeric(kept[column])
        acceptable = value.gt(0).fillna(False)
        stats[column + "_invalid_to_missing"] += int((kept[column].notna() &
~acceptable).sum())
        result[column] = value.where(acceptable)
    stats["scheduled_duration_under_10_retained"] +=
int(result.CRS_ELAPSED_TIME.lt(10).sum())
```

```
    result[TARGET] = delay.loc[valid].ge(15).astype("int8")
    stats["negative_delay_retained"] += int(delay.loc[valid].lt(0).sum())
    stats["delay_over_24h_retained"] += int(delay.loc[valid].gt(1440).sum())
    stats["retained_rows"] += len(result)
    return result[OUTPUT_COLUMNS], stats
```

3. Feature engineering kalender, jam, dan missing

Sumber: `engineer`, baris 158–168 pada versi saat dokumentasi dibuat.

```
# Sumber: e1-data-preparation.py, baris 158–168.
# WHY: Kalender dan jam berulang; akhir siklus berdekatan dengan awal siklus.
# Pasangan sin/cos mewakili kedekatan itu, misalnya pukul 23:59 dengan 00:01.
# Indikator missing membedakan nilai yang tidak tersedia dari nilai yang teramati.
# Waktu missing diberi pasangan (0,0), yang tidak sama dengan titik waktu pada
# lingkaran satuan; ini tidak menyamakan jam kosong dengan pukul 00:00.
# Transformasi memakai periode tetap sehingga tidak perlu mempelajari
validation/test.
# Manfaat representasi siklik terhadap prediksi tetap perlu diuji saat modeling.
def engineer(frame):
    """Transformasi tetap; tidak mempelajari parameter dari data mana pun."""
    frame = frame.copy()
    for column in ["SCHEDULED_DEP_MINUTES", "SCHEDULED_ARR_MINUTES"] + CONTINUOUS:
        frame[column + "_MISSING"] = frame[column].isna().astype("int8")
    for column, (period, offset) in CYCLES.items():
        phase = 2 * np.pi * (frame[column].astype(float) - offset) / period
        # (0, 0) menandai waktu tidak diketahui; flag missing tetap disertakan.
        frame[column + "_SIN"] = np.sin(phase).fillna(0.0)
        frame[column + "_COS"] = np.cos(phase).fillna(0.0)
    return frame[PREPARED_COLUMNS]
```

4. Encoding, imputasi, dan scaling

Sumber: `make_preprocessor`, baris 171–183 pada versi saat dokumentasi dibuat.

```
# Sumber: e1-data-preparation.py, baris 171–183.
# WHY: Jenis fitur berbeda memerlukan transformasi yang berbeda namun konsisten.
# One-hot menghindari urutan numerik buatan pada kode maskapai dan bandara.
# Token missing membedakan kategori kosong; kategori baru dapat ditransformasi
# tanpa error, dengan semua nol pada kelompok one-hot yang tidak dikenal.
# Median mengisi durasi/jarak kosong dengan statistik yang relatif tahan pencilan.
# StandardScaler menyetarakan skala fitur kontinu untuk baseline Logistic
Regression
# dan regularisasinya. Sin/cos yang terbatas pada [-1,1] serta indikator 0/1
diteruskan.
# Pipeline menjaga urutan imputasi lalu encoding/scaling; ColumnTransformer
membatasi
# kolom yang digunakan. Sparse mengurangi penyimpanan nol pada hasil one-hot.
# Fungsi ini hanya menyusun transformer. Fit pada train dilakukan dalam run().
```

```
def make_preprocessor():
    return ColumnTransformer([
        ("category", Pipeline([
            ("imputer", SimpleImputer(strategy="constant",
fill_value="__MISSING__", keep_empty_features=True)),
            ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=True,
dtype=np.float32)),
        ]), CATEGORICAL),
        ("continuous", Pipeline([
            ("imputer", SimpleImputer(strategy="median",
keep_empty_features=True)),
            ("scaler", StandardScaler()),
        ]), CONTINUOUS),
        ("cyclic", "passthrough", CYCLIC_FEATURES),
        ("missing", "passthrough", MISSING_FLAGS),
    ], remainder="drop", sparse_threshold=1.0, verbose_feature_names_out=True)
```

5. Pemilihan masukan model

Sumber: `model_input`, baris 186–190 pada versi saat dokumentasi dibuat.

```
# Sumber: e1-data-preparation.py, baris 186–190.
# WHY: Dataset hasil FE masih memuat metadata dan target untuk audit.
# Daftar MODEL_FEATURES membatasi masukan agar model tidak melihat jawaban
# (DELAYED_15), nomor record, atau tanggal mentah yang bukan fitur yang
disepakati.
# Konversi pd.NA ke np.nan pada kategori menyamakan penanda missing yang diterima
# SimpleImputer. Pemilihan kolom ini tidak mempelajari statistik dari dataset.
def model_input(frame):
    result = frame[MODEL_FEATURES].copy()
    for column in CATEGORICAL:
        result[column] =
result[column].astype(object).where(result[column].notna(), np.nan)
    return result
```

6. Split waktu sebelum fit preprocessing

Potongan ini berada di dalam `run()`. Pembentukan fitur kalender/jam menggunakan aturan tetap; median dan kategori encoder belum dipelajari pada tahap ini. Variabel `source`, `chunk_size`, `prepared`, dan `counts` disiapkan oleh bagian lain dalam skrip utama.

```
# WHY: Membaca kolom yang disetujui saja membatasi masukan dan kebutuhan RAM.
# Chunk menghindari memuat seluruh raw sekaligus; aturan cleaning/FE di sini
tetap.
# Batas tahun memisahkan data masa lalu untuk train dari periode berikutnya.
# Pembagian ini dilakukan sebelum parameter encoding/imputasi/scaling dipelajari.
# Sumber: potongan run() mulai baris 342; konteks folder/variabel ada di skrip
utama.
for frame in pd.read_csv(source, usecols=INPUT_COLUMNS, dtype="string",
```

```
chunksize=chunk_size):
    cleaned, stats = clean_chunk(frame, counts["input_rows"] + 1)
    counts.update(stats)
    engineered = engineer(cleaned)
    for name, (first, last) in SPLITS.items():
        part = engineered.loc[engineered.FL_DATE.between(first, last)]
        part.to_csv(prepared / f"{name}.csv", mode="a", index=False, header=False)
```

7. Fit pada train, transform pada validation/test

Potongan berikut dijalankan di dalam loop subset, dengan train diproses lebih dahulu. `frame` adalah subset yang sudah diurutkan; `preprocessor` dibuat sekali sebelum loop. Bagian penyimpanan dan pencatatan parameter di tengah cabang train disingkat dengan komentar.

```
# WHY: Median, kategori encoder, mean, dan skala harus berasal dari train saja.
# Memakai fit pada validation/test akan memasukkan informasi periode evaluasi.
# Fitur kontinu yang seluruhnya kosong pada train dihentikan karena median tidak
tersedia.
# Transformer yang sama memastikan urutan kolom konsisten antar-subset.
# CSR menyimpan matriks secara sparse; float32 mengurangi ukuran penyimpanan.
# Label disimpan terpisah dengan urutan baris yang sama untuk menjaga pasangan
X/y.
# Pemeriksaan shape dan nilai finite mendeteksi hasil yang belum layak masuk
model.
# Sumber: potongan run() mulai baris 363; bagian pelaporan/penyimpanan di tengah
disingkat.
x = model_input(frame)
if name == "train":
    if any(x[c].isna().all() for c in CONTINUOUS):
        raise ValueError("Ada fitur kontinu yang seluruhnya missing pada train;
median tidak dapat dipelajari.")
    transformed = preprocessor.fit_transform(x)
    feature_names = preprocessor.get_feature_names_out().tolist()
    # ... Penyimpanan artefak dan pencatatan parameter dari run() tidak
ditampilkan.
else:
    transformed = preprocessor.transform(x)
matrix = sparse.csr_matrix(transformed, dtype=np.float32)
del transformed
labels = frame[TARGET].to_numpy(dtype=np.int8)
if matrix.shape != (len(frame), len(feature_names)) or not
np.isfinite(matrix.data).all():
    raise ValueError(f"Matriks fitur tidak valid pada {name}.")
```

Interpretasi

- **Encoding kategori baru:** `handle_unknown="ignore"` menjaga proses berjalan, tetapi kategori baru tidak mendapat bobot khusus yang telah dipelajari. Kelompok one-hot tersebut menjadi nol.

- **Imputasi:** median dipelajari dari train. Nilai yang diisi merupakan pengganti untuk pemodelan, bukan rekonstruksi nilai sebenarnya. Indikator missing mempertahankan informasi bahwa nilai aslinya tidak tersedia.
- **Scaling:** standardisasi mengubah satuan numerik; ini tidak menghapus pencilan dan tidak berarti seluruh fitur akan berada pada rentang 0–1.
- **Sin/cos:** satu pasangan merepresentasikan satu siklus. Representasi ini merupakan pilihan baseline dan belum membuktikan peningkatan akurasi.
- **Pencegahan leakage:** pemilihan fitur dan fit hanya pada train mencegah sumber leakage yang ditangani di sini. Ketersediaan jadwal sebelum keberangkatan tetap menjadi asumsi dataset; jika melakukan cross-validation, transformer harus di-fit kembali pada bagian train setiap fold.
- **Keluaran:** CSV prepared adalah hasil cleaning/FE; matriks X adalah hasil transformasi lengkap. Dokumen ini tidak mengubah dataset, transformer, atau laporan hasil yang sudah tersedia.